Docker details

Tapti Palit

Docker images

- A docker image is a file consisting of all the software it encapsulates
- The file uses a particular type of filesystem known as OverlayFS
  - OverlayFS is a **union filesystem**
  - Can "stack” directories into a single view

Dockerfile and OverlayFS

```
# Use the Ubuntu base image
FROM ubuntu:25.04
```

Image in Dockerhub registry

```
# Set the working directory
WORKDIR /app

ENV DEBIAN_FRONTEND=noninteractive
# Install required dependencies: OpenJDK, Maven, Git
RUN apt-get update && apt-get install -y \
    openjdk-21-jdk \
    maven \
    git \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

# Set JAVA_HOME environment variable
ENV JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
ENV PATH="$JAVA_HOME/bin:$PATH"

# Clone the Java application repository
RUN git clone https://github.com/davsec-
teaching/junit_demo.git .

# Build the application using Maven
RUN mvn clean package

RUN mvn clean test surefire-report:report
# Just run bash
CMD ["bash"]
```

[build artifacts of junit_demo]
junit_demo
openjd-21-jdk; maven; git
ubuntu:25.04

Sample OverlayFS layers

Dockerfile and OverlayFS

```
# Use the Ubuntu base image
FROM ubuntu:25.04
```

Image in Dockerhub registry

```
# Set the working directory
WORKDIR /app

ENV DEBIAN_FRONTEND=noninteractive
# Install required dependencies: OpenJDK, Maven, Git
RUN apt-get update && apt-get install -y \
    openjdk-21-jdk \
    maven \
    git \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

# Set JAVA_HOME environment variable
ENV JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
ENV PATH="$JAVA_HOME/bin:$PATH"

# Clone the Java application repository
RUN git clone https://github.com/davsec-
teaching/junit_demo.git .

# Build the application using Maven
RUN mvn clean package

RUN mvn clean test surefire-report:report
# Just run bash
CMD ["bash"]
```

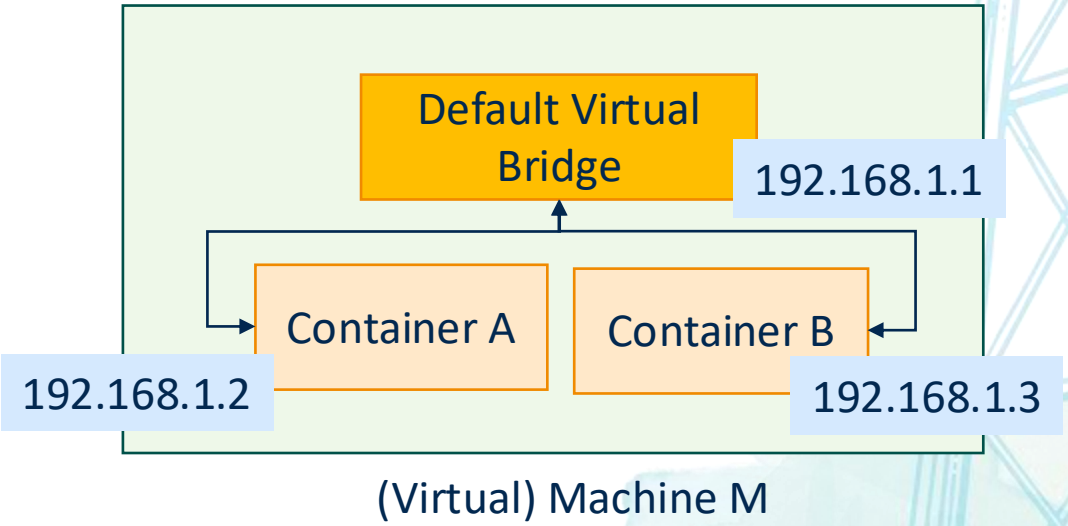
```
# Docker commands

> docker build -t myimage:latest .
> docker images # List all images

> docker run -it myimage:latest  bash # Run the
container

> docker ps # list running containers

> docker ps –a # list all dockers
```

Docker networking

- Default networking (uses virtual bridge)
- Overlay network (uses VXLan)

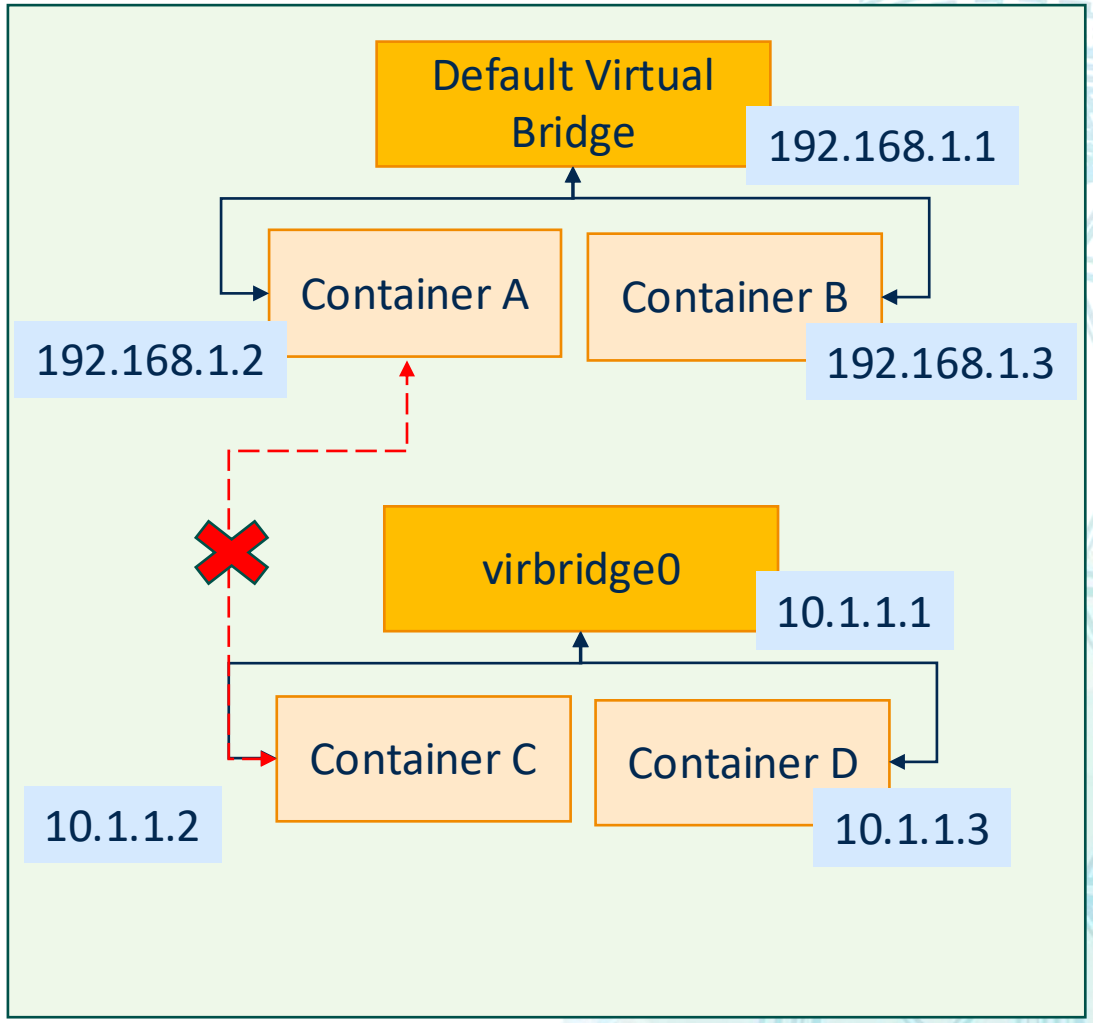
Virtual network bridge

- Docker runtime creates virtual bridges between groups of containers and the host machine
- Each container added to a bridge gets its own IP address endpoint



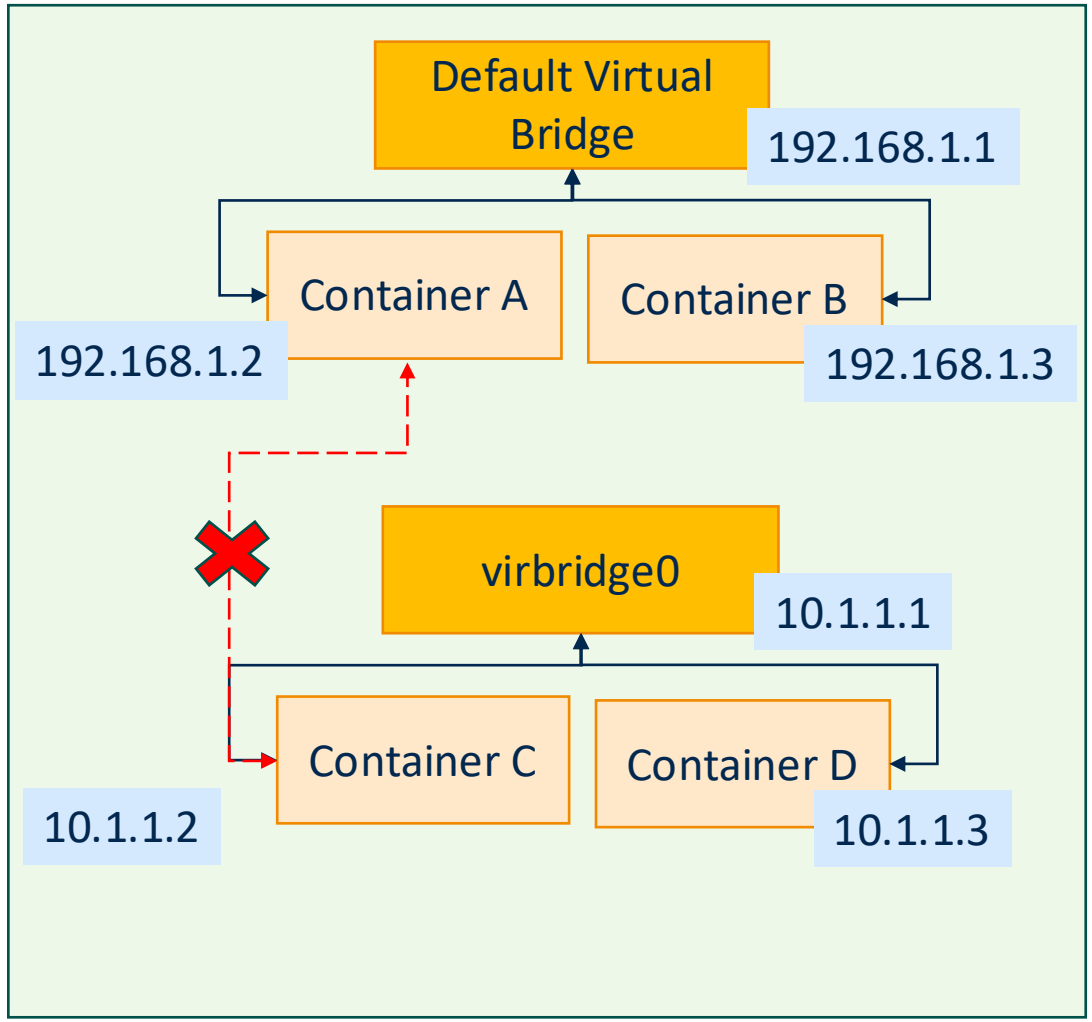
Virtual network bridge

- Docker runtime creates virtual bridges between groups of containers and the host machine
- Each container added to a bridge gets its own IP address endpoint
- Containers on the same bridge can communicate with each other



(Virtual) Machine M

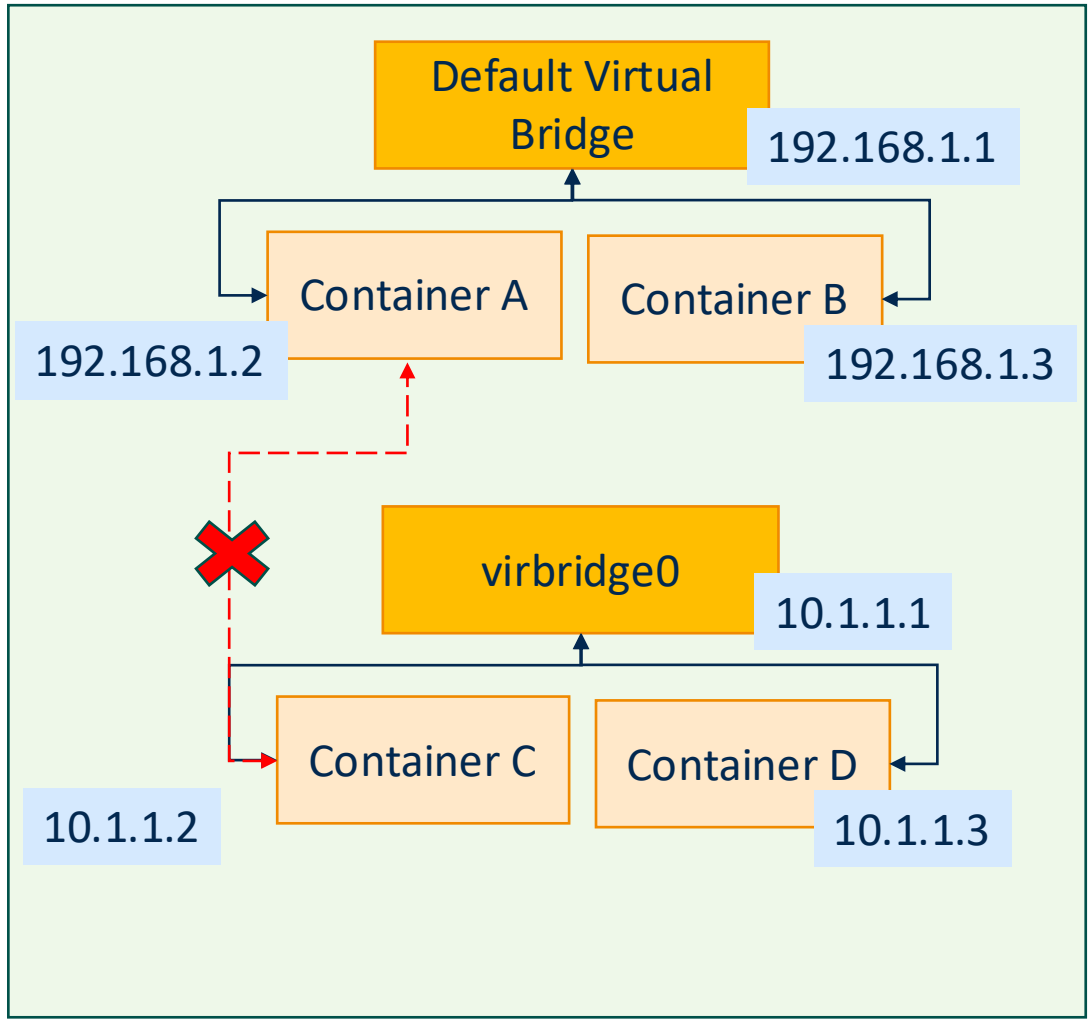
Virtual network bridge



(Virtual) Machine M

- `docker run -it ubuntu:latest /bin/bash --name c-a # On the default bridge`
- `docker run -it ubuntu:latest /bin/bash --name c-b # On the default bridge`
- `ifconfig # show the bridge IP and other info`
- `ping [other_container_ip]`

Virtual network bridge

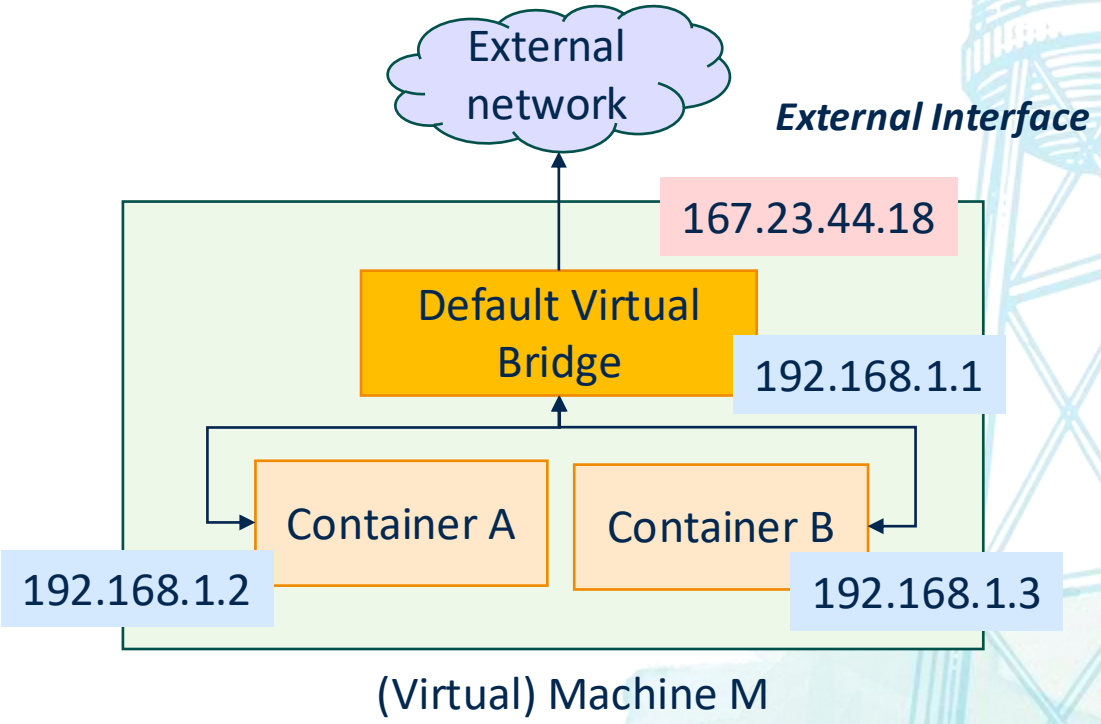


(Virtual) Machine M

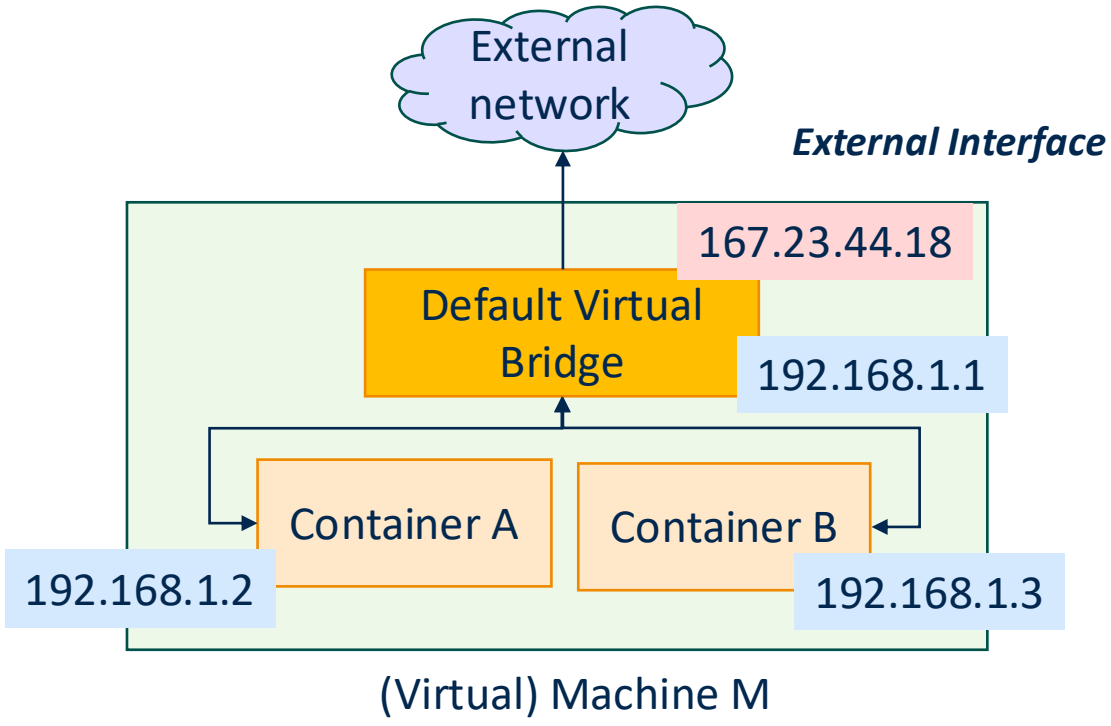
- `docker run -it ubuntu:latest /bin/bash --name c-c --network virbridge0`
- `docker run -it ubuntu:latest /bin/bash --name c-d --network virbridge0`
- `ifconfig # show the bridge IP and other info`
- `ping [other_container_ip]`
- `ping [c-a from c-c]`

Network address translation (NAT)

- Allows outbound connections from inside an internal network
  - Without each internal node having an unique IP
- Also called IP masquerading



NAT - packet send

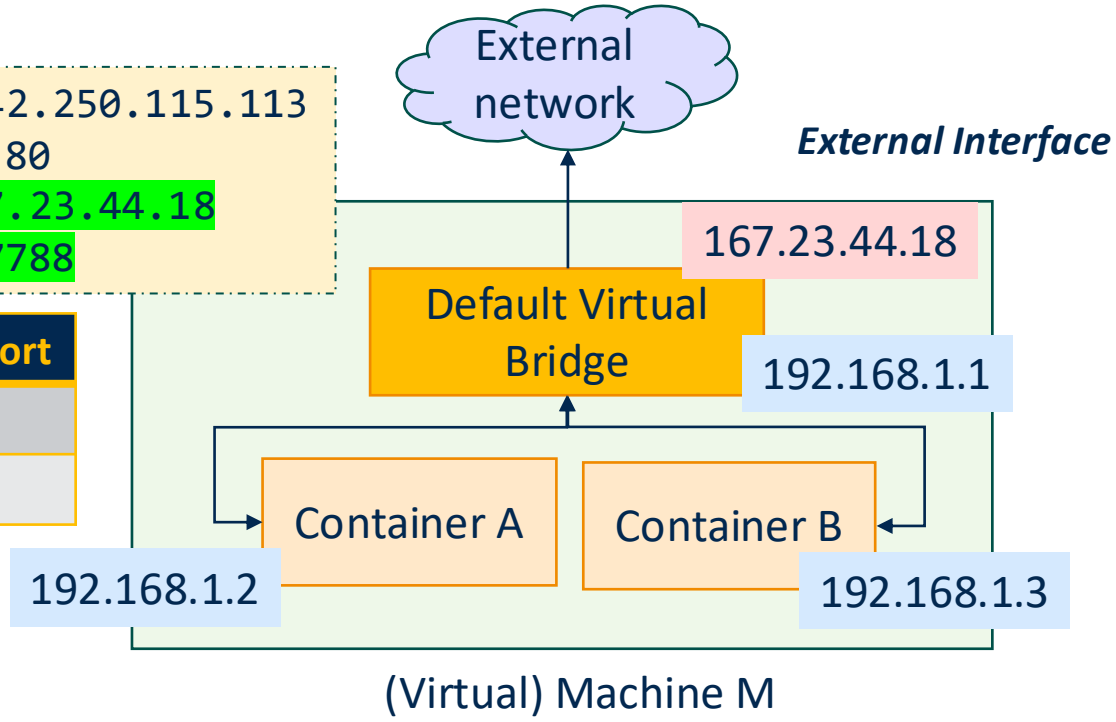



```
DestIP: 142.250.115.113
DestPort: 80
SrcIP: 192.168.1.2
SrcPort: 1192
```

NAT – packet rewrite for send

```
DestIP: 142.250.115.113
DestPort: 80
SrcIP: 167.23.44.18
SrcPort: 7788
```

Bridge Port	Fwd IP	Fwd port
7788	192.168.1.2	1192

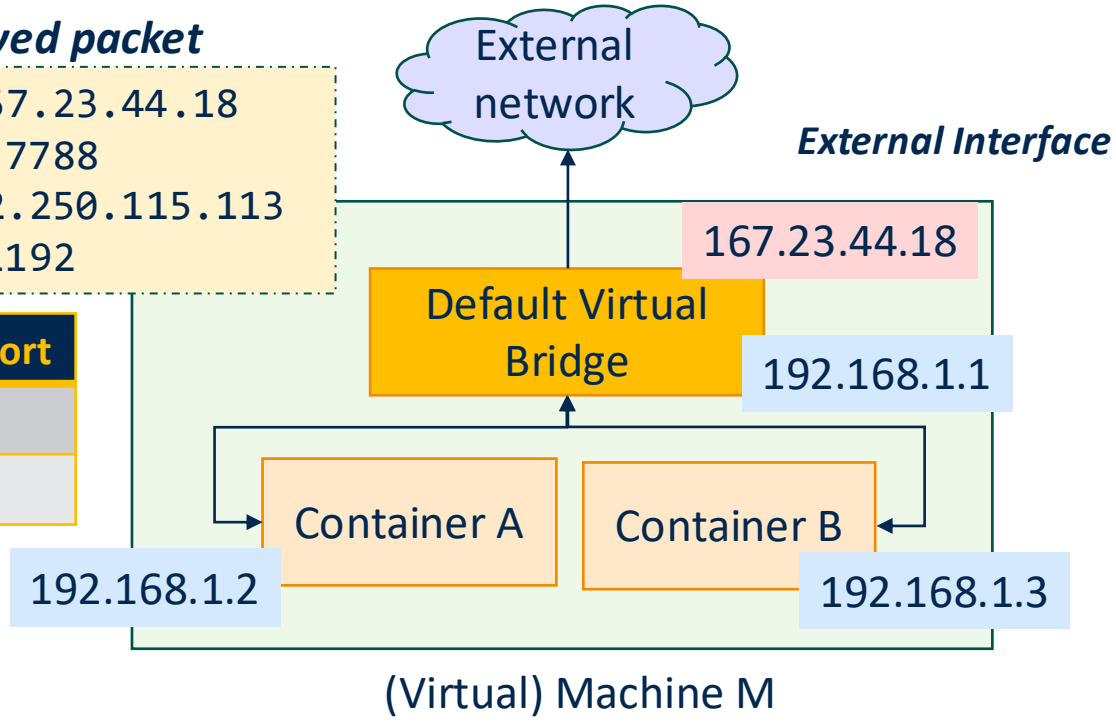


NAT – packet receive

Received packet

```
DestIP: 167.23.44.18
DestPort: 7788
SrcIP: 142.250.115.113
SrcPort: 1192
```

Bridge Port	Fwd IP	Fwd port
7788	192.168.1.2	1192

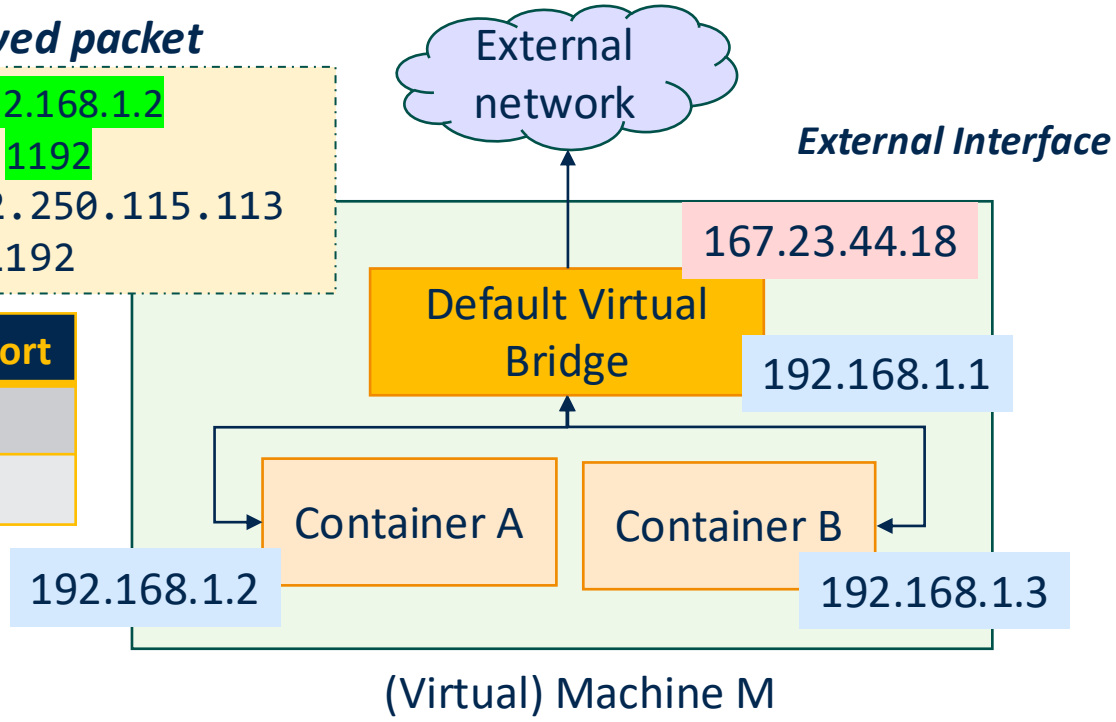


NAT – packet rewrite for receive

Received packet

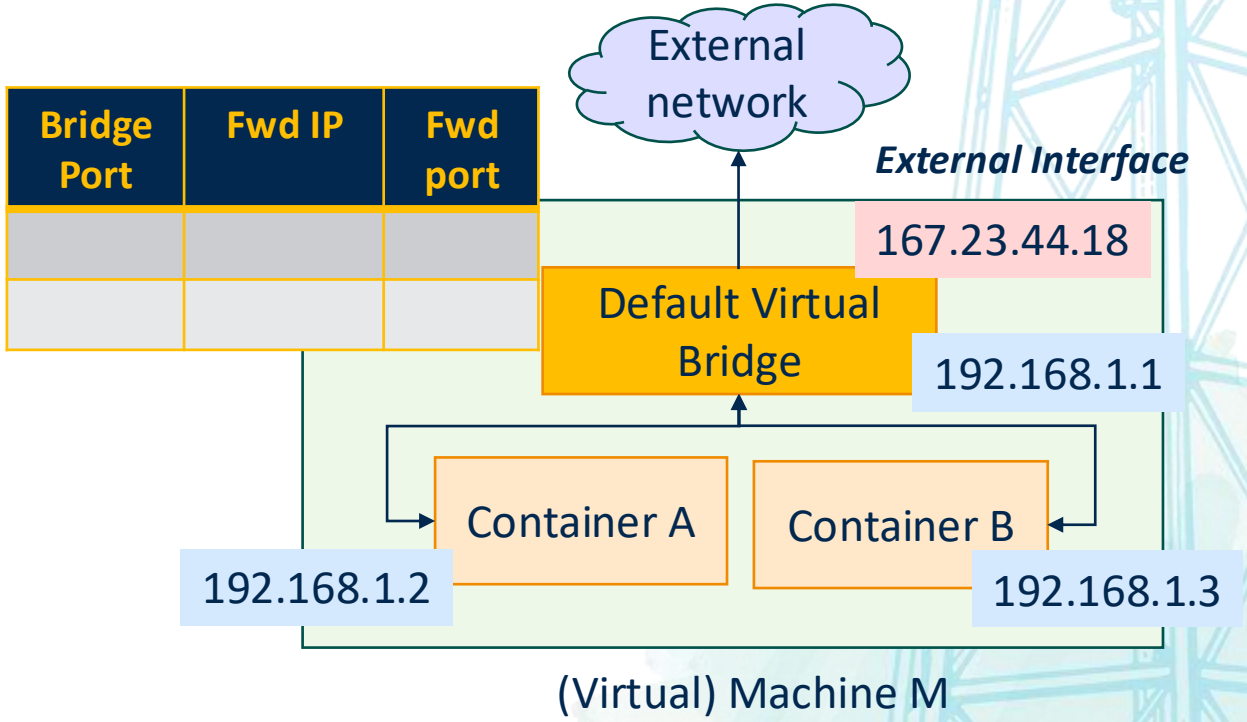
```
DestIP: 192.168.1.2
DestPort: 1192
SrcIP: 142.250.115.113
SrcPort: 1192
```

Bridge Port	Fwd IP	Fwd port
7788	167.23.44.18	1192



Docker default networking

- Every virtual bridge by default uses NAT
- Containers can make outbound connections
- External nodes cannot connect to internal containers



VXLan

- Allowing incoming packets requires VXLan or IP-in-IP
- Core idea - wrap the actual packet (L2/L3) sent by a container in another packet for routing
- VXLan tutorial: https://www.youtube.com/watch?v=YNqKDI_bnPM&list=PLDQaRcbiSnqFe6pyaSy-Hwj8XRFPgZ5h8

Docker volumes

- Docker containers are ephemeral
  - Only the specifications in the Dockerfile persist across respawns
- Can mount volumes to docker containers
- `docker run -v /host/path:/container/path image_name`